



Python Programming — Month 2

## Project 4 — FINAL PROJECT

# To-Do List Manager

Complete Teaching Guide & Documentation

CloudExify Summer Internship 2026

|              |                              |
|--------------|------------------------------|
| Project      | To-Do List Manager           |
| Type         | Productivity CLI Tool        |
| Language     | Python 3.x                   |
| New Concepts | json module, datetime module |
| Difficulty   | Beginner to Intermediate     |
| Hours        | 8–10 hours                   |
| Submission   | End of Month 2 — FINAL       |

## 1. Introduction

The To-Do List Manager is your final project for the CloudExify Python Internship. It introduces two new built-in Python modules — `json` and `datetime` — while building a genuinely useful productivity tool. JSON is a very important concept because it is used everywhere in web development, APIs, and data storage. Learning it now will give you a big advantage.

### What is JSON?

JSON stands for JavaScript Object Notation. It is a way to save Python dictionaries and lists to a file in a structured format. Unlike the plain text files from Month 1, JSON preserves the structure of your data perfectly — making it much easier to save and load complex data.

```
import json

# Save a list of dictionaries to JSON file
tasks = [
    {"id": 1, "title": "Study Python", "done": False},
    {"id": 2, "title": "Build project", "done": True}
]

# SAVE to file:
with open("tasks.json", "w") as f:
    json.dump(tasks, f, indent=4)

# LOAD from file:
with open("tasks.json", "r") as f:
    loaded_tasks = json.load(f)

print(loaded_tasks[0]["title"]) # Study Python
print(loaded_tasks[1]["done"])  # True
```

## 2. Concepts You Will Use

| Concept              | Used For                           |
|----------------------|------------------------------------|
| import json          | Saving and loading tasks as JSON   |
| import datetime      | Recording when task was created    |
| List of dictionaries | Storing all tasks                  |
| Dictionary .get()    | Safely accessing task fields       |
| List comprehension   | Filtering tasks by status/priority |
| sorted() with key=   | Sorting tasks by priority or date  |
| enumerate()          | Displaying task numbers            |
| f-strings            | Formatted table output             |
| Functions            | One function per feature           |

## 3. Task Data Structure

Before writing code, understand exactly what one task looks like:

```
# One task is a dictionary like this:
task = {
    "id": 1,
    "title": "Complete Python project",
    "priority": "High",          # High / Medium / Low
    "due_date": "2026-07-15",   # YYYY-MM-DD format
    "status": "Pending",        # Pending / Done
    "created": "2026-06-01 09:30" # Auto-assigned
}

# All tasks stored in a list:
tasks = []

# Access examples:
print(task["title"])           # Complete Python project
print(task["priority"])        # High
print(task["status"])          # Pending

# Filter only pending tasks:
pending = [t for t in tasks if t["status"] == "Pending"]

# Filter only High priority:
high = [t for t in tasks if t["priority"] == "High"]
```

## 4. Step-by-Step Guide

### Step 1 — Setup and file operations

```
# todo_manager.py
# CloudExify Python Internship – Month 2 Project 4

import json
import os
from datetime import datetime

FILE = "tasks.json"
next_id = 1

def load_tasks():
    global next_id
    if not os.path.exists(FILE):
        return []
    with open(FILE, "r") as f:
        tasks = json.load(f)
    if tasks:
        next_id = max(t["id"] for t in tasks) + 1
    return tasks

def save_tasks(tasks):
    with open(FILE, "w") as f:
        json.dump(tasks, f, indent=4)
```

### Step 2 — Add task function

```
def add_task(tasks):
    global next_id
    print("\n--- ADD NEW TASK ---")

    title = input("Task title: ").strip()
    if not title:
        print("Title cannot be empty!")
        return

    # Priority selection
    print("Priority: 1) High 2) Medium 3) Low")
    while True:
        choice = input("Select (1-3): ").strip()
        if choice == "1": priority = "High"; break
        elif choice == "2": priority = "Medium"; break
        elif choice == "3": priority = "Low"; break
        else: print("Enter 1, 2, or 3!")

    due_date = input("Due date (YYYY-MM-DD) or skip: ").strip()
    if not due_date:
        due_date = "No due date"

    task = {
        "id": next_id,
        "title": title,
        "priority": priority,
        "due_date": due_date,
        "status": "Pending",
        "created": datetime.now().strftime(
            "%Y-%m-%d %H:%M"
        )
    }
    tasks.append(task)
    next_id += 1
    save_tasks(tasks)
    print(f"Task added! ID: {task['id']}")
```

### Step 3 — View tasks function

```
def view_tasks(tasks, filter_status=None,
               filter_priority=None):
    display = tasks

    if filter_status:
        display = [t for t in tasks
                   if t["status"] == filter_status]
    if filter_priority:
        display = [t for t in tasks
                   if t["priority"] == filter_priority]

    if not display:
        print("\nNo tasks found!")
        return

    # Sort: High priority first
    priority_order = {"High": 1, "Medium": 2, "Low": 3}
    display = sorted(display,
                     key=lambda t: priority_order.get(t["priority"], 4))

    print(f"\n{'ID':<5} {'Title':<25} {'Priority':<10}"
          f" {'Status':<10} {'Due Date'}")
    print("-" * 70)

    for t in display:
        status_mark = "DONE" if t["status"] == "Done" else "..."
        print(f"{t['id']:<5} {t['title']:<25} "
              f"{t['priority']:<10} {status_mark:<10} "
              f"{t['due_date']}")
```

## Step 4 — Mark done and delete

```
def mark_done(tasks):
    view_tasks(tasks, filter_status="Pending")

    try:
        task_id = int(input("\nEnter task ID to mark done: "))
    except ValueError:
        print("Please enter a number!")
        return

    for task in tasks:
        if task["id"] == task_id:
            if task["status"] == "Done":
                print("Task is already done!")
            else:
                task["status"] = "Done"
                save_tasks(tasks)
                print(f"Task '{task['title']}' marked as done!")
            return

    print(f"No task found with ID {task_id}")

def delete_task(tasks):
    view_tasks(tasks)

    try:
        task_id = int(input("\nEnter task ID to delete: "))
    except ValueError:
        print("Please enter a number!")
        return

    for i, task in enumerate(tasks):
        if task["id"] == task_id:
            confirm = input(
                f"Delete '{task['title']}'? (yes/no): "
            ).lower()
            if confirm in ["yes", "y"]:
                tasks.pop(i)
                save_tasks(tasks)
                print("Task deleted!")
            return

    print(f"No task found with ID {task_id}")
```

## Step 5 — Statistics and main menu

```
def show_stats(tasks):
    total = len(tasks)
    done = sum(1 for t in tasks if t["status"]=="Done")
    pending = total - done
    high = sum(1 for t in tasks if t["priority"]=="High"
               and t["status"]=="Pending")

    print("\n=== TASK STATISTICS ===")
    print(f"Total Tasks    : {total}")
    print(f"Completed       : {done}")
    print(f"Pending          : {pending}")
    print(f"High Priority    : {high} pending")
    if total > 0:
        pct = (done/total)*100
        print(f"Completion      : {pct:.0f}%")

def main():
    tasks = load_tasks()
    print(f"Loaded {len(tasks)} tasks.")

    while True:
        print("\n=== TO-DO LIST MANAGER ===")
        print("1. Add task")
        print("2. View all tasks")
        print("3. View pending tasks")
        print("4. View high priority")
        print("5. Mark task as done")
        print("6. Delete task")
        print("7. Show statistics")
        print("8. Exit")

        choice = input("Choose (1-8): ").strip()

        if choice=="1": add_task(tasks)
        elif choice=="2": view_tasks(tasks)
        elif choice=="3": view_tasks(tasks,
                                     filter_status="Pending")
        elif choice=="4": view_tasks(tasks,
                                     filter_priority="High")
        elif choice=="5": mark_done(tasks)
        elif choice=="6": delete_task(tasks)
        elif choice=="7": show_stats(tasks)
        elif choice=="8":
            print("Goodbye!"); break
        else: print("Invalid choice!")

if __name__ == "__main__":
    main()
```

## 5. Testing Checklist

| Test Case | Expected Result | Done |
|-----------|-----------------|------|
|-----------|-----------------|------|

|                                           |                         |   |
|-------------------------------------------|-------------------------|---|
| Add task with all fields                  | Task saved to JSON file | ■ |
| Add task with empty title                 | Error message shown     | ■ |
| View all tasks — shows sorted by priority | High first              | ■ |
| View pending only — no done tasks shown   | Filtered correctly      | ■ |
| Mark task as done                         | Status changes, saved   | ■ |
| Mark already-done task                    | Message: already done   | ■ |
| Delete task with confirmation             | Task removed            | ■ |
| Delete task — say no to confirm           | Task kept               | ■ |
| Statistics show correct counts            | Math is accurate        | ■ |
| Restart app — tasks still there           | JSON loads correctly    | ■ |

## 6. Bonus Challenges

| Bonus                                     | Difficulty |
|-------------------------------------------|------------|
| Show overdue tasks (past due date)        | Medium     |
| Edit task title or priority               | Medium     |
| Search tasks by keyword in title          | Easy       |
| Show tasks due today                      | Medium     |
| Add task categories (Work/Study/Personal) | Easy       |

■ HOW TO SUBMIT: 1. Push code to GitHub Repository name: cloudexify-python-p4-yourname  
Must include: main source file + README.md 2. README.md must include: Your name ·  
Registration number · Features list · How to run 3. Include at least 2 screenshots of your running  
program 4. WhatsApp your PM: [CX-2026-XXXX] Project 4 Done — [GitHub Link] FINAL  
SUBMISSION: Include GitHub links for ALL 4 projects in README. Certificate issued within 7 days  
of final submission.

CONGRATULATIONS on completing the CloudExify Python Internship! You built 4 real Python  
projects: Month 1: Expense Tracker + Grade Management System Month 2: Quiz Game + To-Do  
List Manager You now have solid Python foundations and a real portfolio. Top 3 performers win cash  
prizes. All completions get Certificate + LOR. Keep coding. Keep building!

© 2026 CloudExify — Confidential. For CloudExify internship participants only. Redistribution or  
reproduction without written permission is strictly prohibited.